

智能软件工程

当软件工程遇见人工智能

任志磊



大连理工大学

April 9, 2026



目录

1 引言

2 软件工程简史

- 先驱思想
- 起源
- 基础概念
- 关键里程碑
- 现代时期
- LLM 时代
- 结论

3 人工智能简史

- 先驱思想
- 1950s: 领域诞生
- 1960s-70s: 符号 AI 与第一次 AI 寒冬
- 1980s: 专家系统与第二次 AI 寒冬
- 1990s-2000s: 统计转型与机器学习
- 2010s: 深度学习革命
- 2020s: 大语言模型时代
- 结论

4 跨学科结合课题



目录

1 引言

2 软件工程简史

- 先驱思想
- 起源
- 基础概念
- 关键里程碑
- 现代时期
- LLM 时代
- 结论

3 人工智能简史

- 先驱思想
- 1950s: 领域诞生
- 1960s-70s: 符号 AI 与第一次 AI 寒冬
- 1980s: 专家系统与第二次 AI 寒冬
- 1990s-2000s: 统计转型与机器学习
- 2010s: 深度学习革命
- 2020s: 大语言模型时代
- 结论

4 跨学科结合课题



关于我

任志磊

- 大连理工大学教授
- <https://zhilei.ren>
- zren@dlut.edu.cn
- 研究方向：
 - 智能软件工程
 - 软件测试、缺陷定位与自动修复
 - 动态追踪
- <https://github.com/zhileiren/ise-lecture-notes.git>



研究团队

OSCAR

- “**O**perating **S**ystem and **C**ompiler with **A**pplication **R**earch”
- “**O**ptimizing **S**oftware by **C**omputation from **AR**tificial intelligence”
- “**O**perating **S**ystem will **C**rash whenever my **AL**gorithm **R**uns”



讨论内容

内容大纲

- 软件工程与人工智能简介
- 需求工程
- 设计与实现
- 软件测试
- 软件工程研究案例研究



目录

1 引言

2 软件工程简史

- 先驱思想
- 起源
- 基础概念
- 关键里程碑
- 现代时期
- LLM 时代
- 结论

3 人工智能简史

- 先驱思想
- 1950s: 领域诞生
- 1960s-70s: 符号 AI 与第一次 AI 寒冬
- 1980s: 专家系统与第二次 AI 寒冬
- 1990s-2000s: 统计转型与机器学习
- 2010s: 深度学习革命
- 2020s: 大语言模型时代
- 结论

4 跨学科结合课题



先驱思想

谁是第一位程序员？



1843: Ada Lovelace - 第一位程序员

- **Augusta Ada King, Lovelace** 伯爵夫人 (1815-1852)
- 诗人拜伦之女，数学专业

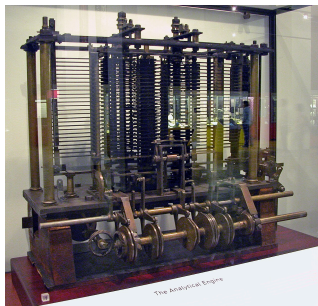


Figure 1: Ada Lovelace 肖像

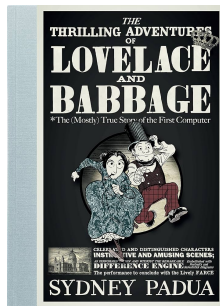


1843: Ada Lovelace - 第一位程序员

- 与 Charles Babbage (查尔斯·巴贝奇) 合作研究其 分析机 (**Analytical Engine**)
- 《Lovelace 与 Babbage 的惊险冒险》(漫画)



(a) 分析机试验模型



(b) 漫画封面

Figure 2: 分析机



1843: Ada Lovelace - 第一位程序员



Figure 3: 蒸汽朋克



1843: Ada Lovelace - 第一位程序员

- 她翻译了一篇关于分析机的意大利文章，并添加了大量的个人注释 (Notes)
- Note G** 包含了被认为是世界上第一个计算机程序的内容

Diagram for the computation by the Engine of the Standard of Bernoulli. See Note G, (page 277 of seq.)

Order of Operation	Variables and Constants	Variables existing in result	Definition of change to the value of any variable	Statement of Result	Units	Working Variables	Result Variables
1	$x_1 = x_1 + 1$	x_1	$x_1 = x_1 + 1$	$x_1 = 1$	1	x_1	
2	$x_2 = x_1 + 1$	x_2	$x_2 = x_1 + 1$	$x_2 = 2$	2	x_2	
3	$x_3 = x_2 + 1$	x_3	$x_3 = x_2 + 1$	$x_3 = 3$	3	x_3	
4	$x_4 = x_3 + 1$	x_4	$x_4 = x_3 + 1$	$x_4 = 4$	4	x_4	
5	$x_5 = x_4 + 1$	x_5	$x_5 = x_4 + 1$	$x_5 = 5$	5	x_5	
6	$x_6 = x_5 + 1$	x_6	$x_6 = x_5 + 1$	$x_6 = 6$	6	x_6	
7	$x_7 = x_6 + 1$	x_7	$x_7 = x_6 + 1$	$x_7 = 7$	7	x_7	
8	$x_8 = x_7 + 1$	x_8	$x_8 = x_7 + 1$	$x_8 = 8$	8	x_8	
9	$x_9 = x_8 + 1$	x_9	$x_9 = x_8 + 1$	$x_9 = 9$	9	x_9	
10	$x_{10} = x_9 + 1$	x_{10}	$x_{10} = x_9 + 1$	$x_{10} = 10$	10	x_{10}	
11	$x_{11} = x_{10} + 1$	x_{11}	$x_{11} = x_{10} + 1$	$x_{11} = 11$	11	x_{11}	
12	$x_{12} = x_{11} + 1$	x_{12}	$x_{12} = x_{11} + 1$	$x_{12} = 12$	12	x_{12}	
13	$x_{13} = x_{12} + 1$	x_{13}	$x_{13} = x_{12} + 1$	$x_{13} = 13$	13	x_{13}	
14	$x_{14} = x_{13} + 1$	x_{14}	$x_{14} = x_{13} + 1$	$x_{14} = 14$	14	x_{14}	
15	$x_{15} = x_{14} + 1$	x_{15}	$x_{15} = x_{14} + 1$	$x_{15} = 15$	15	x_{15}	
16	$x_{16} = x_{15} + 1$	x_{16}	$x_{16} = x_{15} + 1$	$x_{16} = 16$	16	x_{16}	
17	$x_{17} = x_{16} + 1$	x_{17}	$x_{17} = x_{16} + 1$	$x_{17} = 17$	17	x_{17}	
18	$x_{18} = x_{17} + 1$	x_{18}	$x_{18} = x_{17} + 1$	$x_{18} = 18$	18	x_{18}	
19	$x_{19} = x_{18} + 1$	x_{19}	$x_{19} = x_{18} + 1$	$x_{19} = 19$	19	x_{19}	
20	$x_{20} = x_{19} + 1$	x_{20}	$x_{20} = x_{19} + 1$	$x_{20} = 20$	20	x_{20}	
21	$x_{21} = x_{20} + 1$	x_{21}	$x_{21} = x_{20} + 1$	$x_{21} = 21$	21	x_{21}	
22	$x_{22} = x_{21} + 1$	x_{22}	$x_{22} = x_{21} + 1$	$x_{22} = 22$	22	x_{22}	
23	$x_{23} = x_{22} + 1$	x_{23}	$x_{23} = x_{22} + 1$	$x_{23} = 23$	23	x_{23}	
24	$x_{24} = x_{23} + 1$	x_{24}	$x_{24} = x_{23} + 1$	$x_{24} = 24$	24	x_{24}	
25	$x_{25} = x_{24} + 1$	x_{25}	$x_{25} = x_{24} + 1$	$x_{25} = 25$	25	x_{25}	
26	$x_{26} = x_{25} + 1$	x_{26}	$x_{26} = x_{25} + 1$	$x_{26} = 26$	26	x_{26}	
27	$x_{27} = x_{26} + 1$	x_{27}	$x_{27} = x_{26} + 1$	$x_{27} = 27$	27	x_{27}	
28	$x_{28} = x_{27} + 1$	x_{28}	$x_{28} = x_{27} + 1$	$x_{28} = 28$	28	x_{28}	
29	$x_{29} = x_{28} + 1$	x_{29}	$x_{29} = x_{28} + 1$	$x_{29} = 29$	29	x_{29}	
30	$x_{30} = x_{29} + 1$	x_{30}	$x_{30} = x_{29} + 1$	$x_{30} = 30$	30	x_{30}	
31	$x_{31} = x_{30} + 1$	x_{31}	$x_{31} = x_{30} + 1$	$x_{31} = 31$	31	x_{31}	
32	$x_{32} = x_{31} + 1$	x_{32}	$x_{32} = x_{31} + 1$	$x_{32} = 32$	32	x_{32}	
33	$x_{33} = x_{32} + 1$	x_{33}	$x_{33} = x_{32} + 1$	$x_{33} = 33$	33	x_{33}	
34	$x_{34} = x_{33} + 1$	x_{34}	$x_{34} = x_{33} + 1$	$x_{34} = 34$	34	x_{34}	
35	$x_{35} = x_{34} + 1$	x_{35}	$x_{35} = x_{34} + 1$	$x_{35} = 35$	35	x_{35}	
36	$x_{36} = x_{35} + 1$	x_{36}	$x_{36} = x_{35} + 1$	$x_{36} = 36$	36	x_{36}	
37	$x_{37} = x_{36} + 1$	x_{37}	$x_{37} = x_{36} + 1$	$x_{37} = 37$	37	x_{37}	
38	$x_{38} = x_{37} + 1$	x_{38}	$x_{38} = x_{37} + 1$	$x_{38} = 38$	38	x_{38}	
39	$x_{39} = x_{38} + 1$	x_{39}	$x_{39} = x_{38} + 1$	$x_{39} = 39$	39	x_{39}	
40	$x_{40} = x_{39} + 1$	x_{40}	$x_{40} = x_{39} + 1$	$x_{40} = 40$	40	x_{40}	
41	$x_{41} = x_{40} + 1$	x_{41}	$x_{41} = x_{40} + 1$	$x_{41} = 41$	41	x_{41}	
42	$x_{42} = x_{41} + 1$	x_{42}	$x_{42} = x_{41} + 1$	$x_{42} = 42$	42	x_{42}	
43	$x_{43} = x_{42} + 1$	x_{43}	$x_{43} = x_{42} + 1$	$x_{43} = 43$	43	x_{43}	
44	$x_{44} = x_{43} + 1$	x_{44}	$x_{44} = x_{43} + 1$	$x_{44} = 44$	44	x_{44}	
45	$x_{45} = x_{44} + 1$	x_{45}	$x_{45} = x_{44} + 1$	$x_{45} = 45$	45	x_{45}	
46	$x_{46} = x_{45} + 1$	x_{46}	$x_{46} = x_{45} + 1$	$x_{46} = 46$	46	x_{46}	
47	$x_{47} = x_{46} + 1$	x_{47}	$x_{47} = x_{46} + 1$	$x_{47} = 47$	47	x_{47}	
48	$x_{48} = x_{47} + 1$	x_{48}	$x_{48} = x_{47} + 1$	$x_{48} = 48$	48	x_{48}	
49	$x_{49} = x_{48} + 1$	x_{49}	$x_{49} = x_{48} + 1$	$x_{49} = 49$	49	x_{49}	
50	$x_{50} = x_{49} + 1$	x_{50}	$x_{50} = x_{49} + 1$	$x_{50} = 50$	50	x_{50}	
51	$x_{51} = x_{50} + 1$	x_{51}	$x_{51} = x_{50} + 1$	$x_{51} = 51$	51	x_{51}	
52	$x_{52} = x_{51} + 1$	x_{52}	$x_{52} = x_{51} + 1$	$x_{52} = 52$	52	x_{52}	
53	$x_{53} = x_{52} + 1$	x_{53}	$x_{53} = x_{52} + 1$	$x_{53} = 53$	53	x_{53}	
54	$x_{54} = x_{53} + 1$	x_{54}	$x_{54} = x_{53} + 1$	$x_{54} = 54$	54	x_{54}	
55	$x_{55} = x_{54} + 1$	x_{55}	$x_{55} = x_{54} + 1$	$x_{55} = 55$	55	x_{55}	
56	$x_{56} = x_{55} + 1$	x_{56}	$x_{56} = x_{55} + 1$	$x_{56} = 56$	56	x_{56}	
57	$x_{57} = x_{56} + 1$	x_{57}	$x_{57} = x_{56} + 1$	$x_{57} = 57$	57	x_{57}	
58	$x_{58} = x_{57} + 1$	x_{58}	$x_{58} = x_{57} + 1$	$x_{58} = 58$	58	x_{58}	
59	$x_{59} = x_{58} + 1$	x_{59}	$x_{59} = x_{58} + 1$	$x_{59} = 59$	59	x_{59}	
60	$x_{60} = x_{59} + 1$	x_{60}	$x_{60} = x_{59} + 1$	$x_{60} = 60$	60	x_{60}	
61	$x_{61} = x_{60} + 1$	x_{61}	$x_{61} = x_{60} + 1$	$x_{61} = 61$	61	x_{61}	
62	$x_{62} = x_{61} + 1$	x_{62}	$x_{62} = x_{61} + 1$	$x_{62} = 62$	62	x_{62}	
63	$x_{63} = x_{62} + 1$	x_{63}	$x_{63} = x_{62} + 1$	$x_{63} = 63$	63	x_{63}	
64	$x_{64} = x_{63} + 1$	x_{64}	$x_{64} = x_{63} + 1$	$x_{64} = 64$	64	x_{64}	
65	$x_{65} = x_{64} + 1$	x_{65}	$x_{65} = x_{64} + 1$	$x_{65} = 65$	65	x_{65}	
66	$x_{66} = x_{65} + 1$	x_{66}	$x_{66} = x_{65} + 1$	$x_{66} = 66$	66	x_{66}	
67	$x_{67} = x_{66} + 1$	x_{67}	$x_{67} = x_{66} + 1$	$x_{67} = 67$	67	x_{67}	
68	$x_{68} = x_{67} + 1$	x_{68}	$x_{68} = x_{67} + 1$	$x_{68} = 68$	68	x_{68}	
69	$x_{69} = x_{68} + 1$	x_{69}	$x_{69} = x_{68} + 1$	$x_{69} = 69$	69	x_{69}	
70	$x_{70} = x_{69} + 1$	x_{70}	$x_{70} = x_{69} + 1$	$x_{70} = 70$	70	x_{70}	
71	$x_{71} = x_{70} + 1$	x_{71}	$x_{71} = x_{70} + 1$	$x_{71} = 71$	71	x_{71}	
72	$x_{72} = x_{71} + 1$	x_{72}	$x_{72} = x_{71} + 1$	$x_{72} = 72$	72	x_{72}	
73	$x_{73} = x_{72} + 1$	x_{73}	$x_{73} = x_{72} + 1$	$x_{73} = 73$	73	x_{73}	
74	$x_{74} = x_{73} + 1$	x_{74}	$x_{74} = x_{73} + 1$	$x_{74} = 74$	74	x_{74}	
75	$x_{75} = x_{74} + 1$	x_{75}	$x_{75} = x_{74} + 1$	$x_{75} = 75$	75	x_{75}	
76	$x_{76} = x_{75} + 1$	x_{76}	$x_{76} = x_{75} + 1$	$x_{76} = 76$	76	x_{76}	
77	$x_{77} = x_{76} + 1$	x_{77}	$x_{77} = x_{76} + 1$	$x_{77} = 77$	77	x_{77}	
78	$x_{78} = x_{77} + 1$	x_{78}	$x_{78} = x_{77} + 1$	$x_{78} = 78$	78	x_{78}	
79	$x_{79} = x_{78} + 1$	x_{79}	$x_{79} = x_{78} + 1$	$x_{79} = 79$	79	x_{79}	
80	$x_{80} = x_{79} + 1$	x_{80}	$x_{80} = x_{79} + 1$	$x_{80} = 80$	80	x_{80}	
81	$x_{81} = x_{80} + 1$	x_{81}	$x_{81} = x_{80} + 1$	$x_{81} = 81$	81	x_{81}	
82	$x_{82} = x_{81} + 1$	x_{82}	$x_{82} = x_{81} + 1$	$x_{82} = 82$	82	x_{82}	
83	$x_{83} = x_{82} + 1$	x_{83}	$x_{83} = x_{82} + 1$	$x_{83} = 83$	83	x_{83}	
84	$x_{84} = x_{83} + 1$	x_{84}	$x_{84} = x_{83} + 1$	$x_{84} = 84$	84	x_{84}	
85	$x_{85} = x_{84} + 1$	x_{85}	$x_{85} = x_{84} + 1$	$x_{85} = 85$	85	x_{85}	
86	$x_{86} = x_{85} + 1$	x_{86}	$x_{86} = x_{85} + 1$	$x_{86} = 86$	86	x_{86}	
87	$x_{87} = x_{86} + 1$	x_{87}	$x_{87} = x_{86} + 1$	$x_{87} = 87$	87	x_{87}	
88	$x_{88} = x_{87} + 1$	x_{88}	$x_{88} = x_{87} + 1$	$x_{88} = 88$	88	x_{88}	
89	$x_{89} = x_{88} + 1$	x_{89}	$x_{89} = x_{88} + 1$	$x_{89} = 89$	89	x_{89}	
90	$x_{90} = x_{89} + 1$	x_{90}	$x_{90} = x_{89} + 1$	$x_{90} = 90$	90	x_{90}	
91	$x_{91} = x_{90} + 1$	x_{91}	$x_{91} = x_{90} + 1$	$x_{91} = 91$	91	x_{91}	
92	$x_{92} = x_{91} + 1$	x_{92}	$x_{92} = x_{91} + 1$	$x_{92} = 92$	92	x_{92}	
93	$x_{93} = x_{92} + 1$	x_{93}	$x_{93} = x_{92} + 1$	$x_{93} = 93$	93	x_{93}	
94	$x_{94} = x_{93} + 1$	x_{94}	$x_{94} = x_{93} + 1$	$x_{94} = 94$	94	x_{94}	
95	$x_{95} = x_{94} + 1$	x_{95}	$x_{95} = x_{94} + 1$	$x_{95} = 95$	95	x_{95}	
96	$x_{96} = x_{95} + 1$	x_{96}	$x_{96} = x_{95} + 1$	$x_{96} = 96$	96	x_{96}	
97	$x_{97} = x_{96} + 1$	x_{97}	$x_{97} = x_{96} + 1$	$x_{97} = 97$	97	x_{97}	
98	$x_{98} = x_{97} + 1$	x_{98}	$x_{98} = x_{97} + 1$	$x_{98} = 98$	98	x_{98}	
99	$x_{99} = x_{98} + 1$	x_{99}	$x_{99} = x_{98} + 1$	$x_{99} = 99$	99	x_{99}	
100	$x_{100} = x_{99} + 1$	x_{100}	$x_{100} = x_{99} + 1$	$x_{100} = 100$	100	x_{100}	

Figure 4: 第一个程序



Lovelace 的遗产

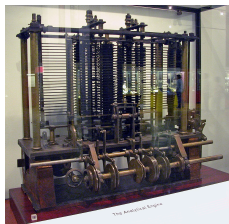
- **Ada** 编程语言：1980 年由美国国防部以她的名字命名。设计用于大规模、安全关键型系统。
- **Ada Lovelace** 日：每年十月的第二个星期二举行的国际性庆祝活动。



Figure 5: Ada 编程语言吉祥物



从愿景到现实



1840s Lovelace 的理论程序

↓ 100 年 ↓



1940s ENIAC - 第一台电子计算机



高级语言的黎明

- **1950s 背景：**使用机器码/汇编进行编程既繁琐又容易出错
- **愿景：**创建更易于人类阅读且与机器无关的语言
- **出现的两条路径：**
 - **Fortran (1957)：**为科学计算设计 - 命令式
 - **Lisp (1958)：**为人工智能研究设计 - 函数式



左：John Backus, Fortran 创始人；右：John McCarthy, Lisp 创始人



Fortran：第一门高级语言

- 名称：公式翻译系统 (FORmula TRANslating system)
- 创建者：1957 年由 IBM 的 John Backus 创建
- 主要目标：使科学计算更简单、更高效
- 关键创新：
 - 历史上第一个编译器
 - 类似于代数的数学符号
 - 数组操作和复数处理
 - 令人惊讶的高效代码生成
- 影响：彻底改变了科学和工程计算



Fortran Code Example

Listing 1: Fortran 77 Style

C AREA OF A TRIANGLE WITH HERON'S FORMULA

```
PROGRAM TRIANGLE
REAL A, B, C, AREA, S

READ *, A, B, C
S = (A + B + C) / 2.0
AREA = SQRT(S * (S - A) * (S - B) * (S - C))

PRINT *, 'AREA = ', AREA
END PROGRAM
```

- 命令式风格：显式的操作序列
- 数学焦点：公式的直接翻译
- 简单数据流：变量被原地修改
- 固定格式：早期版本要求特定的列位置



Lisp: 第二门高级语言

- 名称: 列表处理 (LISt Processing)
- 创建者: 1958 年由 MIT 的 John McCarthy 创建
- 主要目标: 人工智能研究
- 关键创新:
 - 函数式编程范式
 - 链表作为基本数据结构
 - 垃圾回收机制
 - 同质性 (Homoiconicity): 代码即数据, 数据即代码
 - 递归作为主要控制结构
- 影响: 人工智能研究和函数式编程的基础



Lisp 代码示例

Listing 2: Common Lisp Style

```
;; 使用递归计算阶乘
(defun factorial (n)
  (if (<= n 1)
      1
      (* n (factorial (- n 1)))))

;; 在列表上映射函数
(defun square-list (numbers)
  (mapcar #'(lambda (x) (* x x)) numbers))

;; 使用高阶函数
(let ((numbers '(1 2 3 4 5)))
  (print (factorial 5))
  (print (square-list numbers)))
```

- 函数式风格：强调表达式而非语句
- 递归思维：通过递归分解问题
- 列表操作：内置的列表处理操作



范式对比：命令式 vs 函数式

- **Fortran (命令式范式):**
 - 哲学：“如何计算” - 步骤序列
 - 状态变化：变量被修改
 - 控制结构：循环、条件判断、goto
 - 主要抽象：过程和子程序
 - 内存管理：手动/基于栈
- **Lisp (函数式范式):**
 - 哲学：“计算什么” - 表达式与变换
 - 不可变性：尽可能避免改变状态
 - 控制结构：递归、函数组合
 - 主要抽象：函数作为一等公民
 - 内存管理：自动垃圾回收



设计哲学对比

- **Fortran** 的实用导向:
 - 专为数值计算效率而设计
 - 紧密映射数学符号
 - 针对数组操作和线性代数进行了优化
 - 最小化语法，最大化计算能力
 - 目标用户：科学家和工程师
- **Lisp** 的理论基础:
 - 基于 Lambda 演算和递归函数理论
 - 拥抱符号计算和人工智能问题
 - 通过宏 (Macros) 实现强大的元编程
 - 通过 REPL 环境进行交互式开发
 - 目标用户：AI 研究员和计算机科学家



软件工程的黎明

- **1968 NATO 会议**：“软件工程”一词正式提出
- 解决“软件危机” - 软件开发日益增长的复杂性和成本
- 核心目标：将工程原则应用于软件开发



Figure 6: 1968 NATO 加米施会议



1968: “Goto 语句被认为有害”

- **Edsger Dijkstra** 的开创性信函 (1968)
- 主张反对使用 GOTO 语句
- 推动结构化编程原则
- 现代控制结构 (if-else, while, for) 的基础
- 强调代码的可读性和可维护性

Dijkstra 的洞察

程序测试可以用来展示 Bug 的存在，但永远无法展示它们的缺失。



Dijkstra 论自然语言编程 (1978)

- Dijkstra 强烈反对“用自然语言告诉计算机做什么”的设想。
 - 自然语言具有高度的模糊性 (**Ambiguity**)。
 - 编程的本质是严谨的思维过程。如果一个人能够清晰严谨地用自然语言描述问题，他其实已经在编写一种形式化语言。

“所谓自然语言的‘自然性’，归根结底，不过是我们能轻松地用它说出那些荒谬性并不显而易见的话。” ——*E.W. Dijkstra*



Dijkstra 论自然语言编程 (1978)



Figure 7: 那就是代码

当代回响

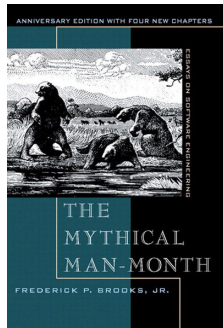
- 2026 年，LLM 让“自然语言编程”成为现实：Karpathy 几乎不再亲手写码，Stripe 每周合并 1300 个 Agent PR
- 但 AI 编程实践的教训与 Dijkstra 的警告惊人地吻合：
 - ① 甜蜜期：动动嘴就能跑出 Demo 接受“不用想太清楚就能开始”
 - ② 阵痛期：AI 习惯性“漏需求”，架构松散，上下文一长就“降智”
 - ③ 回归期：最有效的流程变成了：描述意图 → 转化为 Spec → AI 分析排除矛盾 → 人类审查 → 测试套件 → 分拆执行

这个流程的每一步，本质上都是在把自然语言的“毛坯”，精加工成形式化的“构件”。



1975: 人月神话

- **Frederick Brooks** 的经典著作 (1975)
- 基于管理 IBM OS/360 开发的经验
- 核心论点：“向进度落后的软件项目添加人力会使其更慢”
- **布鲁克斯法则**：由于沟通成本，在软件工作中“人月”是一个危险且错误的度量单位。
- “没有银弹”



1986: 没有银弹 (No Silver Bullet)

- **Fred Brooks** 的核心观点：在十年内，没有任何单一的技术或管理进展能让软件生产率提高一个数量级。
- 两种复杂度：
 - **本质复杂度 (Essential Complexity)**：软件所要解决的业务逻辑、数据关系和算法本身的复杂性。
 - **附属性复杂度 (Accidental Complexity)**：由于开发环境、语言语法、陈旧工具等引起的非必要困难。
- 结论：大多数现代进步（如高级语言、IDE）只解决了附属性复杂度。本质复杂度无法通过“魔法”消除。



LLM 时代的“银弹”讨论

LLM 是终极银弹吗？

- 附属性复杂度的终结：LLM 极大地消除了语法记忆、API 查找和样板代码编写的负担（接近零附属性复杂度）。
- 本质复杂度的回归：即使代码生成速度快了 10 倍，需求定义、系统架构设计和逻辑正确性（本质复杂度）依然由人类承担。
- 新挑战：AI 幻觉引入了新的测试和验证成本。



观点演变总结

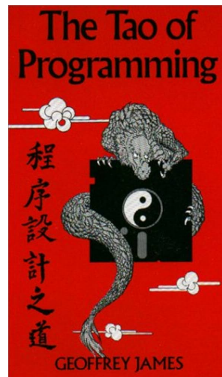
核心议题	经典观点 (Brooks/Dijkstra)	LLM 时代特点
生产力瓶颈	本质复杂度 (逻辑、沟通) 不可逾越。	LLM 加速了实现过程, 但沟通和逻辑确认仍是瓶颈。
编程媒介	必须使用严谨的数学/形式化语言。	自然语言成为第一接口, 但代码仍作为底层 “真理”。
程序员角色	逻辑构建者与细节实现者。	审查者 (Reviewer) 与 需求策展人 (Curator)。

Table 1: 经典理论在 AI 时代的延续与转变



编程之道

- 一位经理找到大师级程序员，向他展示了一个新应用的需求文档。经理问大师：“如果我分配五个程序员来设计这个系统，需要多长时间？”
- “需要一年，”大师不假思索地答道。
- “但是我们现在就需要这个系统，甚至越快越好！如果我分配十个程序员，需要多长时间？”
- 大师级程序员皱了皱眉头。
- “那样的话，需要两年。”
- “那如果我分配一百个程序员呢？”
- 大师级程序员耸了耸肩。
- “那么设计将永远无法完成，”他说。



编程之道

- 道生机器语言，机器语言生汇编语言。
- 汇编语言生编译器。至今已有万种语言。
- 每种语言各有所用，无论多么卑微。
- 每种语言都表达了软件的阴阳。每种语言在道中皆有其位。
- 但如果可以避免，请不要用 COBOL 编程。

道德经

- 道生一，一生二，二生三，三生万物。
- The Tao gave birth to One, One gave birth to Two, Two gave birth to Three, Three gave birth to ten thousand things.



Figure 9: 《道德经》帛书（公元前 2 世纪）



1984: 关于信任信任的反思

- **Ken Thompson** 的图灵奖演讲 (1984)
- 编译器可以插入能够自我传播的后门
- 软件供应链安全的基础性洞察



Figure 10: Ken Thompson 和 Dennis Ritchie 与 UNIX

核心启示

我们用来构建软件的工具本身必须是可信的。没有终极的信任根。

1984: 关于信任信任的反思

- **Ken Thompson** 的图灵奖演讲 (1984)
- 编译器可以插入能够自我传播的后门
- 软件供应链安全的基础性洞察

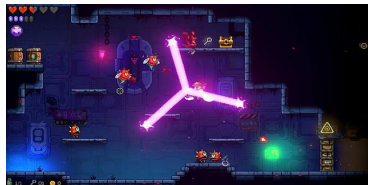


Figure 10: Ken Thompson 和 Dennis Ritchie 与 UNIX

核心启示

我们用来构建软件的工具本身必须是可信的。没有终极的信任根。

RogueLike



Rogue

Rogue 是迷宫探索式电子游戏，最早由迈克尔·托依和格伦·韦科曼在 1980 年左右开发¹。部分因为游戏内容的过程生成，游戏在 1980 年代中期大学 Unix 系统上很流行。Rogue 使迷宫探索在电子游戏领域普及，其他开发者制作了诸多统称为“类 rogue”（Roguelike）的派生作品。比如它直接给与了 Hack 灵感，此游戏之后又衍生出 NetHack。类 rogue 还影响了其他类型的商业游戏，如暗黑破坏神 Rogue 在当时的计算机研究者和大学生之间引发轰动，甚至 Unix 之父之一的肯·汤普逊也沉迷其中。丹尼斯·里奇开玩笑道：“Rogue 就是史上最浪费 CPU 时钟周期的东西（the biggest waste of CPU cycles in history）”

¹<https://zh.wikipedia.org/wiki/Rogue>



RogueLike

游戏玩法核心元素在 2008 年的柏林国际 Roguelike 游戏开发大会得到了明确的定义，称之为“柏林准则”

- 通过随机生成地牢来增强可玩性
- 游戏使用永久死亡机制
- 游戏是回合制的
- 游戏不应该有过多限制
- 游戏应该为玩家提供完成同样方式的多种不同方法
- 为了生存玩家必须妥善管理自己的资源
- 游戏核心内容应是 hack and slash
- 游戏要求玩家探索地图，寻找宝藏并杜绝背板



Hack and NetHack

Hack (1982 年) 是杰·芬拉森 (Jay Fenlason) 在肯尼·伍德蓝德 (Kenny Woodland)、麦克·托米 (Mike Thome) 和强纳森·沛恩 (Jonathan Payne) 的帮助下开发的。他们曾找到 Rogue 的制作者托伊和阿诺, 希望能获取游戏源代码, 却遭到拒绝, 这迫使他们从原有的游戏草稿设计一个完整的游戏出来。

麦克·史蒂芬森 (Mike Stephenson) 接手 Hack 的代码维护工作后, 他按照美国宾夕法尼亚大学哲学教授伊兹查克·米勒和电脑极客珍妮特·瓦兹 (Janet Walz) 的建议改进了这款游戏。这三位组建了一个“开发组”, 着手对 Hack 的源代码做重大修改。部分因为他们的工作主要通过 USENET 来沟通, 他们称修改后的版本为 NetHack, Net 便是指该游戏通过网络开发。



NetHack

- NetHack 是一款最初在 1987 年发布的 Roguelike 单人游戏，拥有由 ASCII 字符组成的图形界面。它继承了 Hack (1985 年) 及更早的 Rogue (1980 年)。²
- 游戏名字中的“网络”(Net)是指开发过程通过互联网合作完成。
- 而“Hack”是指角色扮演游戏的特点——hack and slash。
- 玩家扮演一个地下城探险者寻找 Yendor 的项链。
- 玩家需要选择自己所扮演的角色并指定性别、种族、职业和阵营，或者选择让系统随机产生一个角色。游戏者可以扮演经典奇幻角色，比如骑士、野蛮人、巫师、游侠、女武神、僧侣和武士，也可以选择一些比较少见的角色，诸如考古学家、游客和洞穴人。

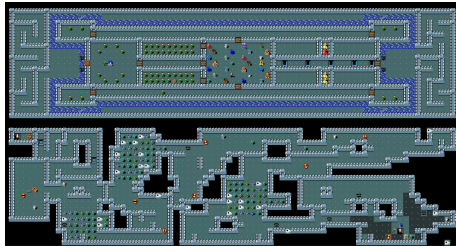
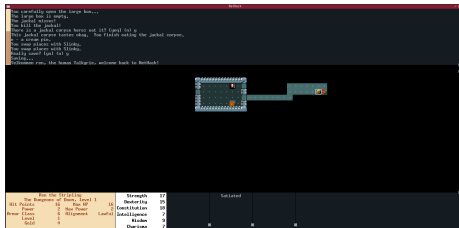
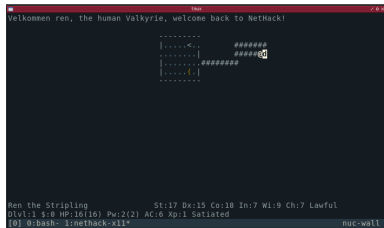
²<https://zh.wikipedia.org/wiki/NetHack>



DnD



Nethack



1980s-1990s: GNU 与开源革命

- **Richard Stallman** 发起 **GNU** 项目 (1983)
- 自由软件基金会 (FSF) 成立 (1985)



Figure 11: Richard 与 GNU 项目



1980s-1990s: GNU 与开源革命

GNU: GNU's Not Unix



1980s-1990s: GNU 与开源革命

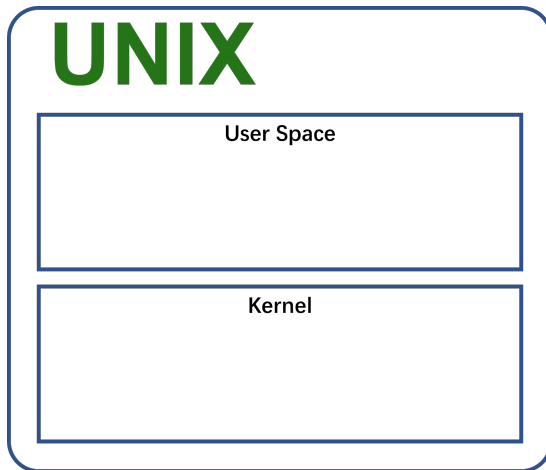


Figure 12: GNU 项目



1980s-1990s: GNU 与开源革命

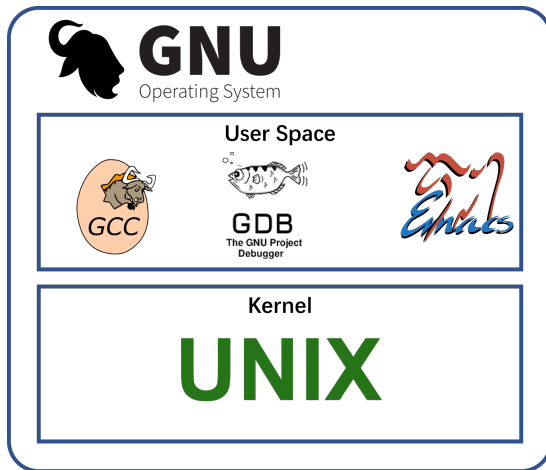


Figure 13: GNU 项目



1980s-1990s: GNU 与开源革命

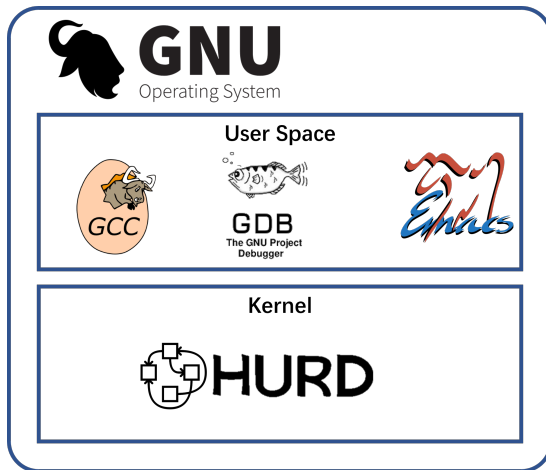


Figure 14: GNU 项目



1980s-1990s: GNU 与开源革命

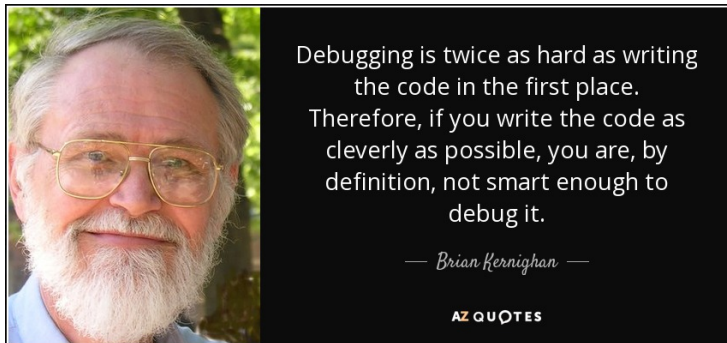


Figure 15: Brian Kernighan 谈调试



1980s-1990s: GNU 与开源革命

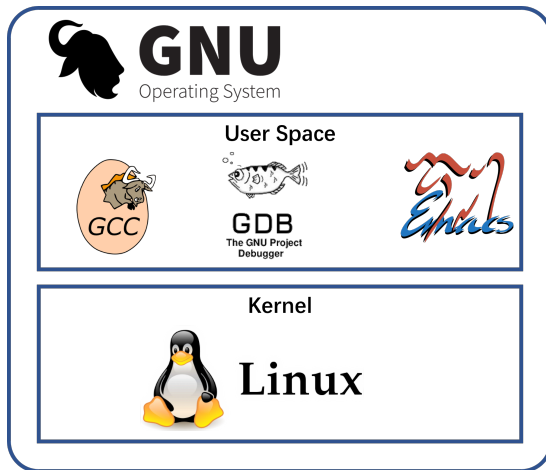


Figure 16: GNU 项目



1980s-1990s: GNU 与开源革命

- Linus Torvalds 创建 Linux 内核 (1991)
- 开源促进会 (OSI) 成立 (1998)

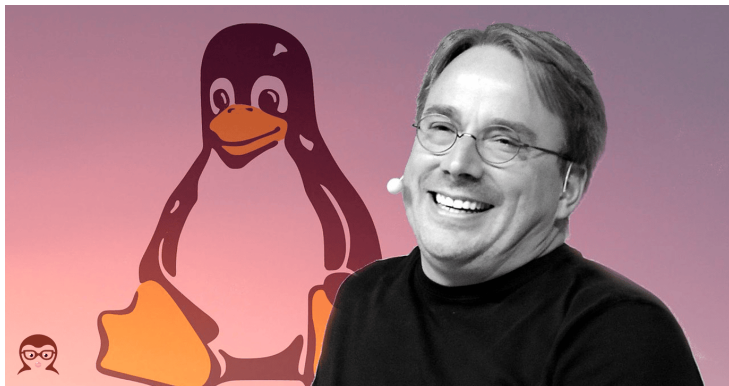


Figure 17: Linus Torvalds 与 Linux



从 Netscape 到 Firefox

巨人的陨落：Netscape 的“大教堂”

- **Netscape Navigator** 在 1990 年代中期曾占据 90% 的市场份额。
- 在“第一次浏览器大战”中输给了微软的 Internet Explorer。
- 面对被淘汰的风险，Netscape 在 **1998** 年采取了激进的一步。



从 Netscape 到 Firefox



(a)

(b)

Figure 18: 大教堂与市集 (The Cathedral and the Bazaar)



从 Netscape 到 Firefox

激进转变：拥抱“市集”

- 1998 年 1 月, Netscape 将其浏览器代码 (Netscape Communicator 4.0) 开源, 以利用分布式开发者的力量。
- 成立了 **Mozilla** 组织来管理该项目。
- 旧代码臃肿难用; 开发者决定从零开始, 使用新的 **Gecko** 渲染引擎。



从 Netscape 到 Firefox

凤凰涅槃：Firefox 的诞生

- 从 Mozilla 代码库中，诞生了一个名为 **Phoenix** (2002) 的轻量级、以用户为中心的浏览器。
- 经过商标争议 (Phoenix → Firebird → **Firefox**)，Mozilla Firefox 1.0 于 **2004** 年发布。
- Firefox 挑战了 IE 的垄断，捍卫了开放标准和用户选择权。



Figure 19: Firefox Logo 演变



哲学引擎：大教堂 vs. 市集

大教堂模式

- 封闭、集中的开发。
- 由少数专门的专家团队构建。
- 版本发布周期长，是大规模的事件。
- 例子：1998 年之前的 Netscape，传统软件开发。

市集模式

- 开放、分散的开发。
- 代码通过互联网公开开发。
- “只要眼睛足够多，所有 Bug 都会无所遁形。”
- 例子：Mozilla, Linux, Firefox。

核心启示

从 Netscape 到 Firefox 的故事表明，通过全球协作，管理良好的 市集模式能够比封闭模式更有效地构建复杂的高质量系统。



遗产与深远影响

- 技术基础：Gecko 引擎为 Firefox 铺平了道路。Netscape 的遗产还包括 JavaScript, SSL 和 Cookie。
- 机制化：**Mozilla** 基金会 (成立于 2003 年 7 月) 确保了项目使命超越了单一公司的寿命。
- 持久战：Firefox 成为 IE 的主要替代品，证明了开源软件对主流用户的可行性。

从大教堂灰烬中诞生的开源市集，帮助确保了互联网的开放性和竞争性。



趣味冷知识

- Tøyen Cola: <https://distrita.com/toyen-cola-is-a-speciality-in-oslo-nor>
- Open Soda: <http://www.opensoda.org/>
- Cube Cola: <https://cube-cola.org/>



Figure 20: 开源可乐



趣味冷知识

- 开源硬件由开放设计运动所设计和提供的技术实体组成
- 硬件设计（如机械图、电路图、物料清单、PCB 布局数据、HDL 源码和集成电路布局数据）以及驱动硬件的软件，全部在自由/开放条款下发布

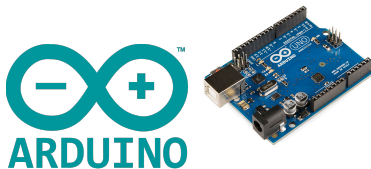


Figure 21: 开源硬件



1980s-1990s: 面向对象革命

- **Smalltalk** (1970s-1980s): 纯面向对象 (OOP) 概念
- **C++** (1985): 将 OOP 引入系统级编程
- **Java** (1995): “一次编写，到处运行”
- 核心原则: 封装、继承、多态
- 设计模式 (Gang of Four 著作, 1994)

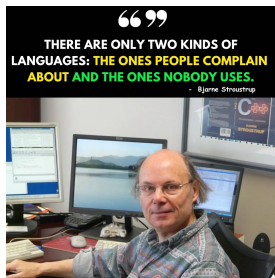


Figure 22: C++ 创始人的语录



2000s: 敏捷与 DevOps

- 敏捷宣言 (2001): 强调迭代开发和客户协作
- **DevOps** 运动 (2000 年代末)
- 弥合开发与运维的鸿沟, 以 Git 和 CI/CD 为特征
- 基础设施即代码 (IaC)

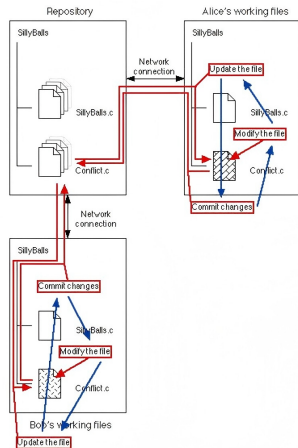


Figure 23: 敏捷 12 条原则



1990s: CVS - 并发版本系统

- 开发于 **1986** 年，在 1990 年代占据主导地位
- 集中式架构：单一中央仓库
- 关键功能：
 - 基础版本追踪
 - 支持多开发者
 - 基于网络的访问
- 局限性：
 - 缺乏原子提交
 - 分支/合并功能差
 - 网络操作缓慢
 - 单点故障风险



CVS 集中式架构

CVS 将版本控制带给了大众，但协作过程是顺序且谨慎的。



2000: Subversion - “正确的 CVS”

- **2000** 年创建，旨在修复 CVS 的局限性
- 相比 **CVS** 的改进：
 - 原子提交：更改要么全部成功，要么全部失败
 - 更好的分支/标记：廉价的复制操作
 - 版本化的目录和重命名
 - 更好的二进制文件处理
- 依然是集中式：单一仓库模型
- 采用情况：迅速成为企业标准
- 局限性：中央服务器 = 单点故障，离线工作困难



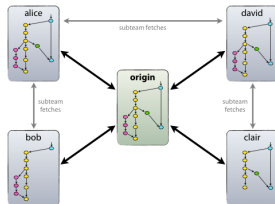
2002-2005: BitKeeper 争议

- **Linux** 内核开发规模超出了基于补丁的工作流
- **BitKeeper**: 专有的分布式版本控制系统
- **2002** 年被 **Linux** 采用: 实现了真正的分布式开发
- 革命性功能:
 - 分布式仓库
 - 卓越的分支/合并功能
 - 快速的性能
- 争议 (**2005**): 许可协议纠纷导致 BitKeeper 停止免费提供
- 催化剂: 迫使 Linux 社区创建自己的解决方案



2005: Git - Linux 社区的回答

- 2005 年由 **Linus Torvalds** 创建，紧随 BitKeeper 争议之后
 - 分布式且点对点
 - 强大的加密完整性 (SHA-1)
 - 快速的分支/合并
 - 高效处理大型项目
 - 支持非线性开发
- 关键创新：
 - 内容寻址存储
 - 基于有向无环图 (DAG) 的历史记录
 - 三阶段思维 (工作目录、暂存区、仓库)



xkcd 谈 Git

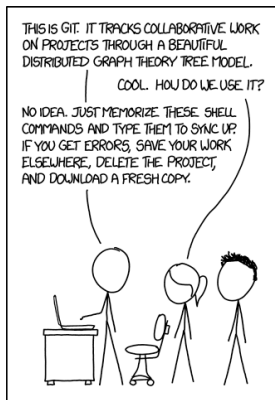


Figure 24: Git 漫画



为什么 Git 赢了：分布式工作流

集中式 (CVS/SVN)

- 单一信任源
- 顺序提交
- 必须联网
- 分支成本高
- 线性历史

分布式 (Git)

- 每个克隆都是完整仓库
- 并行开发
- 离线工作，随后同步
- 廉价分支
- 非线性历史

游戏规则改变者

Git 实现了诸如 功能分支 (**feature branching**) 和 拉取请求 (**pull requests**) 的工作流，使开源协作具有可扩展性。



GitHub 与社交编程革命 (2008)

- **GitHub 2008 年上线**：为 Git 添加了社交层
- **关键创新**：
 - 拉取请求 (Pull Request)：代码审查成为一等公民
 - Fork & Pull 模型：轻松贡献任何项目
 - Issue、项目看板、Wiki：集成化的项目管理
- **网络效应**：使开源协作成为主流
- **企业采用**：GitHub Enterprise, GitLab, Bitbucket

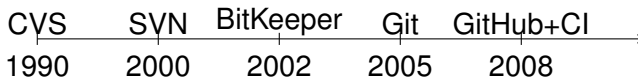


Git 赋能的 CI/CD：自动化革命

- 持续集成 (CI)：自动构建并测试每一次提交
- 持续部署/交付 (CD)：自动部署通过测试的代码
- **Git 作为触发器：**
 - 推送到仓库触发自动化流水线
 - 分支保护规则强制执行质量关卡
 - 拉取请求状态检查防止引入 Bug
- 关键工具：Jenkins, GitHub Actions, GitLab CI, CircleCI



演变时间线



- **1990s:** 集中式时代 - 顺序协作
- **2000s:** 集中式改良 - 更好但仍受限
- **2002-2005:** 分布式觉醒 - BitKeeper 展示了方向
- **2005+:** Git 革命 - 并行、分布式协作
- **2008+:** 社交编程 + 自动化 - CI/CD 生态



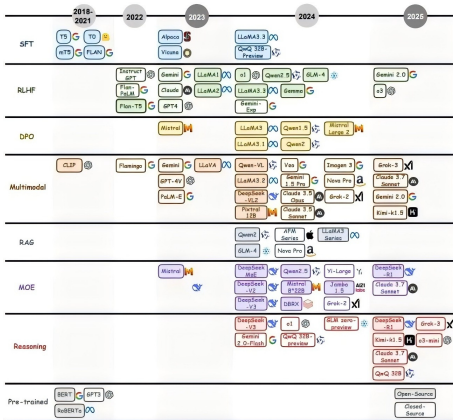
2010s: AIOps 与云原生

- **AIOps:** 将人工智能应用于运维
 - 自动化监控和事件响应
 - 预测系统性能分析
 - 异常检测与根因分析
- 云原生开发
 - 微服务架构
 - 容器化 (Docker, Kubernetes)
 - 无服务器计算 (Serverless)



- 大语言模型改变软件工程
- GitHub Copilot (2021) 及类似工具

A SURVEY ON POST-TRAINING OF LARGE LANGUAGE MODELS



LLM 对软件工程的影响

- 代码生成：从自动补全到生成整个函数
- 调试辅助：AI 驱动的 Bug 检测与修复
- 文档自动化：自动生成文档
- 代码审查：AI 辅助的质量保证
- 测试：智能测试用例生成

未来方向

- AI 结对编程成为标准
- 新的编程范式正在出现
- 重心转向提示工程 (Prompt Engineering) 和 AI 监管



展望未来

- 从严谨的工程化到 AI 增强的持续演进
- 越来越强调自动化和智能化
- 人为因素在系统设计和监管中依然至关重要
- AI 很强大，但它既不是银弹也不是万灵药

旅程仍在继续...



目录

1 引言

2 软件工程简史

- 先驱思想
- 起源
- 基础概念
- 关键里程碑
- 现代时期
- LLM 时代
- 结论

3 人工智能简史

- 先驱思想
- 1950s: 领域诞生
- 1960s-70s: 符号 AI 与第一次 AI 寒冬
- 1980s: 专家系统与第二次 AI 寒冬
- 1990s-2000s: 统计转型与机器学习
- 2010s: 深度学习革命
- 2020s: 大语言模型时代
- 结论

4 跨学科结合课题



构思的种子

- **Ada Lovelace (1843):** 质疑机器是否能产生原创想法，还是只能执行人类的命令。
- **20 世纪早期:** Karel apek 的剧本《R.U.R.》(1920) 创造了“robot”（机器人）一词。



Figure 26: Capek 的剧本 R.U.R



1950s: 奠基的十年

- Alan Turing (1950): 发表《计算机器与智能》，提出 图灵测试。

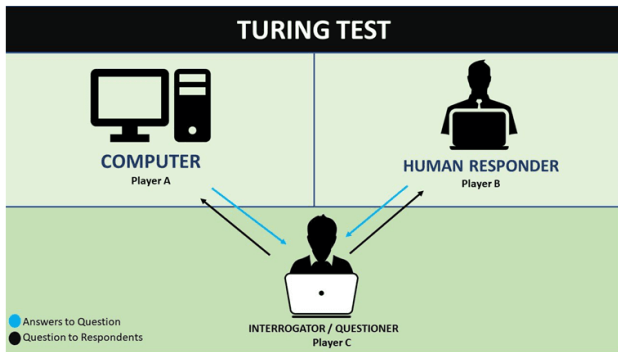


Figure 27: 图灵测试示意图



1950s: 奠基的十年

- 术语诞生: John McCarthy 组织了 达特茅斯会议 (1956) 并创造了“人工智能 (Artificial Intelligence)”一词。
- 设定目标: “学习的每一个方面或智能的任何其他特征, 在原则上都可以被精确地描述, 从而可以制造出一台机器来模拟它。” - 达特茅斯提案



Figure 28: 达特茅斯会议



1960s-70s: 符号 AI 及其局限

- **符号 AI (GOF AI):** 专注于操作符号和逻辑规则来表示知识并解决问题。
- **ELIZA (1966):** 一个早期的自然语言处理程序，模拟罗杰斯心理治疗师。
- **符号 AI** 在处理现实世界的复杂性和歧义时遇到困难。它需要大规模的手动编码知识库。
- **Lighthill 报告 (1973):** 批评 AI 未能实现其宏伟承诺，导致资金大幅削减 - 第一次 AI 寒冬。



LLAMA3.2 拜访 Emacs Doctor

```
File Edit Options Buffers Tools Text Help
I am the psychotherapist. Please, describe your problems. Each time
you are finished talking, type RET twice.

I've been experiencing recurring nightmares and flashbacks from a traumatic event in my childhood that I've tried to suppress
for years.

Is it because you have tried to suppress for years that you came to
me?

Yes, feeling overwhelmed by the memories has become unbearable and I'm hoping therapy can help me process and heal from them.

Why do you say that?

Because the nightmares are getting more intense and my anxiety is affecting every aspect of my life, including my relationships
and work.

-UUU:**- F1 *doctor* Top L1 (Doctor Fill) -----
Beginning of buffer

ren@ren-NUC12WSH17:~$ ollama run llama3.2
>>> Suppose you are a patient visiting a psychotherapist. Each time only say one short sentence briefly. Please, describe your problem
... S
I've been experiencing recurring nightmares and flashbacks from a traumatic event in my childhood that I've tried to suppress
for years.

>>> Is it because you have tried to suppress for years that you came to
... me?
Yes, feeling overwhelmed by the memories has become unbearable and I'm hoping therapy can help me process and heal from them.

>>> Why do you say that?
Because the nightmares are getting more intense and my anxiety is affecting every aspect of my life, including my relationships
and work.

>>> Send a message (/? for help)

[1] 0:emacs* nuc-wall
```

Figure 30: LLAMA3.2 拜访 Emacs Doctor



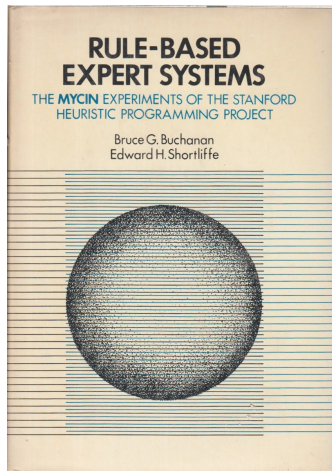
1980s: 专家系统的兴衰

专家系统

- 旨在通过基于规则的系统捕获人类专家的知识。
- 用于诊断血液感染的 MYCIN 系统表现与专家相当。

第二次 AI 寒冬

- 专家系统非常 **脆弱**，维护成本高，且不具备学习能力。
- 它们无法扩展到狭窄领域之外。
- 到了 1980 年代末，炒作周期再次破裂，导致 **第二次 AI 寒冬**。



基于规则的专家系统



1990s-2000s: 新范式

- 从逻辑向统计的转变：研究人员不再编程逻辑规则，而是专注于创建能够从数据中学习的系统（如 SVM、随机森林）。
- **IBM Deep Blue (1997)**：击败了世界象棋冠军加里·卡斯帕罗夫。这是符号 AI 的里程碑成就，主要依靠搜索和评估函数。



Figure 31: 深蓝 vs. 加里·卡斯帕罗夫



2010s: 深度学习爆发

突破点

- 关键促成因素：海量数据集（大数据）和强大的并行计算能力（GPU）。
- 技术：深度学习使用多层神经网络，从原始数据中自动学习层次化特征。

里程碑成就

- **AlphaGo (2016)**: 在围棋比赛中击败世界冠军，围棋曾被认为是人类智力游戏的最后堡垒。

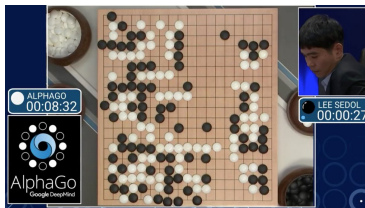


Figure 32: AlphaGo vs. 李世石



2020s: LLM 与生成式 AI 时代

- 规模即一切：发现大幅增加神经网络规模（大小、数据、计算量）会导致涌现能力的产生。
- **Transformer 架构 (2017)**：成为几乎所有现代 AI 的基础模型，实现了序列（如文本）的并行处理。
- **大语言模型 (LLMs)**：
 - 像 GPT-4, Claude, Llama 这样的模型是在互联网大部分公开数据上训练的。
 - 能够生成类人文本、翻译、总结和编写代码。
- **生成式 AI**：LLM 驱动了 ChatGPT 等工具，将 AI 能力带给大众。
- **多模态 AI**：能够理解和生成不同模态（文本、图像、音频）的模型。



AI 历史：浪潮之旅

AI 领域经历了乐观的浪潮和随后的“寒冬”，每一次浪潮都在前人的基础上不断发展。

演进模式

符号 AI (规则) → 统计机器学习 (数据) → 深度学习 (表示) → 生成式 AI (创作)



展望未来

- 通用人工智能 (AGI): 人类水平智能的最初目标仍在远方。
- 伦理与安全: 随着 AI 变得强大, 关于偏见、控制和与人类价值对齐的担忧变得至关重要。
- AI 普及化: AI 能力正在集成到几乎每一个软件产品中。
- 共生: 未来涉及人类与 AI 的紧密协作, 增强人类的能力。

制造思考机器的梦想比计算机本身更久远。这段旅程远未结束。



目录

1 引言

2 软件工程简史

- 先驱思想
- 起源
- 基础概念
- 关键里程碑
- 现代时期
- LLM 时代
- 结论

3 人工智能简史

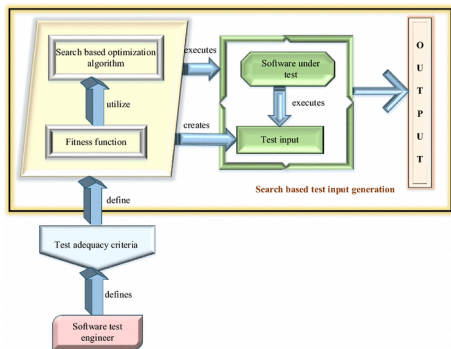
- 先驱思想
- 1950s: 领域诞生
- 1960s-70s: 符号 AI 与第一次 AI 寒冬
- 1980s: 专家系统与第二次 AI 寒冬
- 1990s-2000s: 统计转型与机器学习
- 2010s: 深度学习革命
- 2020s: 大语言模型时代
- 结论

4 跨学科结合课题



基于搜索的软件工程

基于搜索的软件工程 (SBSE) 将元启发式搜索技术 (如遗传算法、模拟退火和禁忌搜索) 应用于软件工程问题。软件工程中的许多活动都可以被表述为优化问题。由于计算复杂性或结构假设, 运筹学的优化技术通常不适用于大规模软件工程问题。研究者使用对问题结构假设极少的元启发式搜索技术, 来寻找近优解或“足够好”的解。



挖掘软件仓库

在软件工程中，挖掘软件仓库 (MSR) 领域分析软件仓库中的丰富数据 (如版本控制库、邮件列表归档、缺陷追踪系统等)，以发现关于软件系统、项目和软件工程的有意义且可操作的信息。

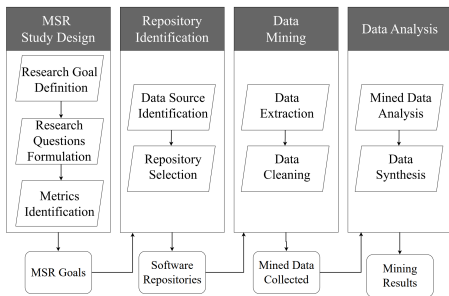


Figure 34: 典型的 MSR workflow



变更何时会引入修复？

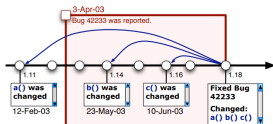


Figure 2: Locate fix-inducing changes for bug 42233

```
$ cvs annotate -r 1.17 Foo.java
...
19: 1.11 (john 12-Feb-03): public int a() {
20: 1.11 (john 12-Feb-03):     return i/0;
...
39: 1.10 (mary 12-Jan-03): public int b() {
40: 1.14 (kate 23-May-03):     return 42;
...
59: 1.10 (mary 17-Jan-03): public void c() {
60: 1.16 (mary 10-Jun-03):     int i=0;
...
```

(a) 将事务链接到缺陷报告

	Day of Week						
% of revisions	Mon	Tue	Wed	Thu	Fri	Sat	avg
$P(\text{fix})$	18.4	20.9	20.0	22.3	24.0	14.7	16.9
$P(\text{bug})$	11.3	10.4	11.1	12.1	12.2	11.7	11.6
$P(\text{bug} \cap \text{fix})$	4.6	4.8	4.6	5.2	5.6	4.5	4.9
$P(\neg \text{bug} \cap \neg \text{fix})$	74.9	73.5	73.5	70.8	63.4	78.1	72.7
$P(\text{bug} \text{fix})$	25.1	22.9	23.3	23.5	23.2	30.3	26.4
$P(\text{bug} \neg \text{fix})$	8.2	7.1	8.1	8.8	8.7	8.4	8.6

Table 5: Distribution of fixes and fix-inducing changes across day of week in ECLIPSE

	Day of Week						
% of revisions	Mon	Tue	Wed	Thu	Fri	Sat	avg
$P(\text{fix})$	42.5	46.5	49.7	45.9	48.4	50.2	48.5
$P(\text{bug})$	39.1	44.1	41.2	40.8	46.2	44.9	41.5
$P(\text{bug} \cap \text{fix})$	19.4	23.6	22.8	21.6	26.9	19.6	21.9
$P(\neg \text{bug} \cap \neg \text{fix})$	37.8	33.0	31.9	34.9	32.3	24.5	31.9
$P(\text{bug} \text{fix})$	45.7	50.8	45.8	47.1	55.6	39.1	45.2
$P(\text{bug} \neg \text{fix})$	34.1	38.3	36.7	35.5	37.3	50.6	38.1

Table 6: Distribution of fixes and fix-inducing changes across day of week in MOZILLA

(b) 修复和修复诱导变更的分步

Figure 35: 关于周五的研究



为什么不要在周五写代码 (by Gemini)

理由 1：周末是神圣的

- 生产故障风险：周五下午部署的 Bug 可能意味着整个周末都要进行紧急修复。
- “多米诺效应”：一个看似微小的问题可能连锁反应，需要多人协作解决。
- 压力与倦怠：持续的周末加班是开发者职业倦怠的主要原因。

理由 2：人为因素

- 人员可用性低：关键人员（开发、测试、运维、支持）可能不在岗。
- 响应速度慢：召集合适的团队解决问题需要更长时间。
- 周末疲劳：周五的专注力自然下降，增加了出错的可能性。



主动方案：只读周五 (Read-Only Fridays)

- 将重心从部署新功能转向规划和准备。
- 活动包括：
 - 代码审查
 - 编写文档
 - 规划下周迭代
 - 学习与专业发展
- 目标：在进入周末前保持生产环境的稳定和可预测性。



实证软件工程

实证软件工程 (ESE) 是研究软件工程现象的子领域。该现象可以指软件开发工具、技术、实践、流程或组织因素。ESE 常用的研究方法包括：

- ① 初级研究（实验、案例研究、调查研究、模拟等）
- ② 次级研究（系统综述、系统映射研究等）



UML 的实践现状

TABLE I. DECLARED CURRENT UML USE AMONG 50 PROFESSIONAL SOFTWARE DEVELOPERS FROM 50 COMPANIES.

Category of UML Use	Instances of Declared Current Use
no UML	35
retrofit	1
automated code generation	3
selective	11
wholehearted	0

(a) 声明的当前 UML 使用情况

TABLE II. ELEMENTS OF UML USED BY THE 11 'SELECTIVE' USERS.

UML diagrams	Number of users	Reported to be used for...
Class diagrams	7	structure, conceptual models, concept analysis of domain, architecture, interfaces
Sequence diagrams	6	requirements elicitation, eliciting behaviors, instantiation history
Activity diagrams	6	modeling concurrency, eliciting useful behaviors, ordering processes
State machine diagrams	3	
Use case diagrams	1	represent requirements

(b) 11 位“选择性”用户使用的 UML 图

Figure 36: UML 在实践中 (ICSE 2013)

实践中的草图与图表 (FSE 2014)

大多数草图和图表是非正式的。40% 根本不包含 UML 元素，48% 包含部分元素，仅 9% 完全由 UML 元素组成。受访者指出，即使使用了 UML，通常也没有严格遵守标准定义。



参考文献

- [1] NATO Software Engineering Conference (1968)
- [2] Dijkstra, E. W. (1968). Go To Statement Considered Harmful
- [3] Brooks, F. P. (1975). *The Mythical Man-Month*. Addison-Wesley.
- [4] Thompson, K. (1984). Reflections on Trusting Trust
- [5] Stallman, R. (1983). GNU Manifesto
- [6] Beck, K. (2001). Agile Manifesto
- [7] Chen, M. et al. (2023). Evaluating Large Language Models Trained on Code
- [8] Iivarski J, Zimmermann T, Zeller A. When do changes induce fixes?. 2005.
- [9] Petre M. UML in practice. ICSE 2013.
- [10] Baltes S, Diehl S. Sketches and diagrams in practice. FSE 2014.

